

Spis treści

1. Wprowadzenie i kontekst problemu	2
2. Cel analizy i pytania decyzyjne.....	3
3. Opis case study i założeń	4
4. Praca z bazą danych	5
4.1. Implementacja	7
4.2. Analiza Danych – Zapytania SQL	9
5. Power BI – narzędzie raportowania	12
5.1. Dlaczego Power BI?	12
5.2. Tworzenie Tabeli Zdenormalizowanej	14
5.3. Wizualizacje.....	16
6. Wnioski	19

1. Wprowadzenie i kontekst problemu

Współczesne organizacje gromadzą ogromne ilości danych w tradycyjnych relacyjnych bazach danych, które są zoptymalizowane pod kątem systemów OLTP (On-Line Transactional Processing). Ich celem jest wprowadzanie i przechowywanie danych transakcyjnych w wysokim stopniu normalizacji, co skutkuje rozproszeniem informacji pomiędzy wieloma zależnymi od siebie tabelami.

Przez to struktura staje się niewystarczająca dla zaawansowanej analityki Business Intelligence. Konieczna jest więc transformacja danych do schematu odpowiedniego dla systemu OLAP (On-Line Analytical Processing).

Proces ten opiera się na denormalizacji poprzez wyodrębnienie centralnej tabeli faktów zawierającej mierzalne fakty biznesowe oraz otaczających ją tabel wymiarów z atrybutami opisowymi.

2. Cel analizy i pytania decyzyjne

Celem projektu było przekształcenie tej bazy transakcyjnej w system analityczny typu OLAP (Online Analytical Processing), który umożliwia zadawanie złożonych pytań biznesowych:

- Który kraj generuje największą sprzedaż?
- Które kategorie produktów są najpopularniejsze?
- Jak zmieniała się sprzedaż w czasie?

3. Opis case study i założeń

Baza Northwind jest systemem OLTP zatem rejestruje każde zamówienie, produkt, klienta w znormalizowanej strukturze tabel (nastawienie na szybkość zapisu, nie analizę).

Cechy OLTP:

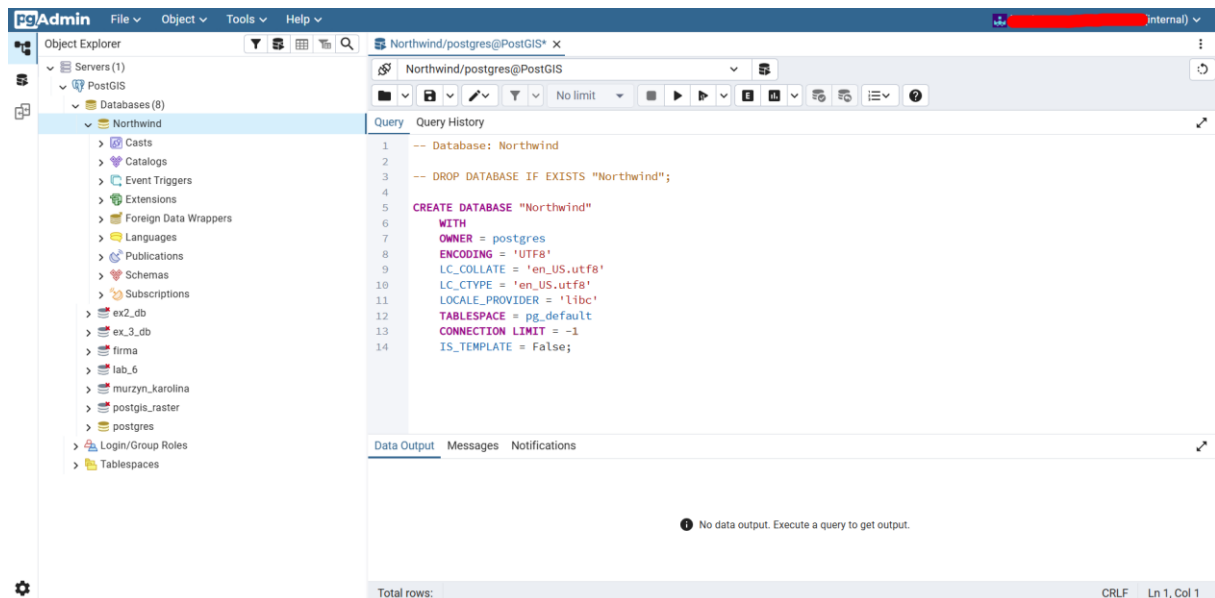
- Duża liczba krótkich transakcji (INSERT, UPDATE, DELETE)
- Dane znormalizowane (3NF) - brak redundancji
- Optymalizacja pod kątem zapisu

Celem było stworzenie systemu OLAP który jest dedykowany do tworzenia analiz dla dużych zbiorów przy jednoczesnym uproszczeniu zapytań SQL, mniejszym użyciu pamięci a następnie stworzenie wizualizacji.

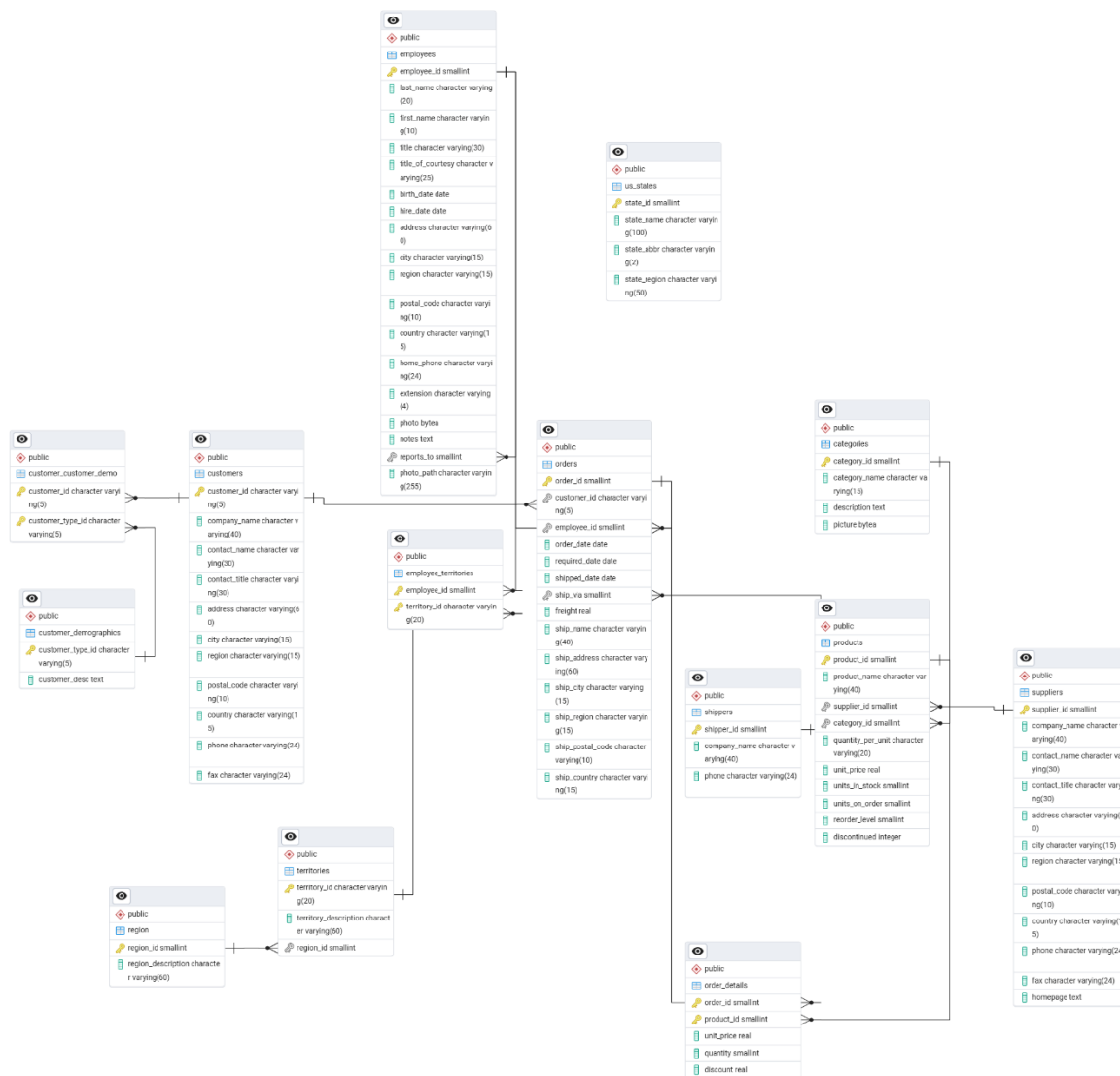
Cecha	Baza Danych (OLTP)	Hurtownia Danych (OLAP)
Cel (Goal)	Wprowadzanie i przechowywanie danych	Odczyt i analiza danych
Procesowanie	OLTP (transakcyjne)	OLAP (analityczne)
Zakres (Scope)	Pojedyncza aplikacja	Cały dział lub organizacja
Struktura	Płaska i relacyjna	Ustrukturyzowana, wielowymiarowa
Tabele i łączenie	Złożone (normalizacja)	Bardzo ułatwione (denormalizacja)
Projektowanie	Metoda E-R (Entity-Relationship)	Schemat gwiazdy lub płatka śniegu
Aktualność	Czas rzeczywisty	Okresowa (cykle ETL)

4. Praca z bazą danych

Dane zostały wczytane ze skryptu pobranego z repozytorium <https://github.com/microsoft/sql-server-samples/tree/master/samples/databases/northwind-pubs> oraz wczytane za przy pomocy systemu zarządzania bazą danych PostgreSQL.



Gotowa architektura OLTP przedstawia 13 tabel źródłowych będących w znormalizowanej strukturze trzeciej postaci normalnej (3N). Proces biznesowy jest przede wszystkim tworzony przez tabele: orders, order_details, products, customers, employees oraz categories i suppliers.



Na nową architekturę został wybrany schemat gwiazdy, wynikało to m. in. z:

- Prostszej struktury zapytań analitycznych (JOIN do tabel wymiarowych),
- Lepszej wydajności dla narzędzi BI i agregacji OLAP.

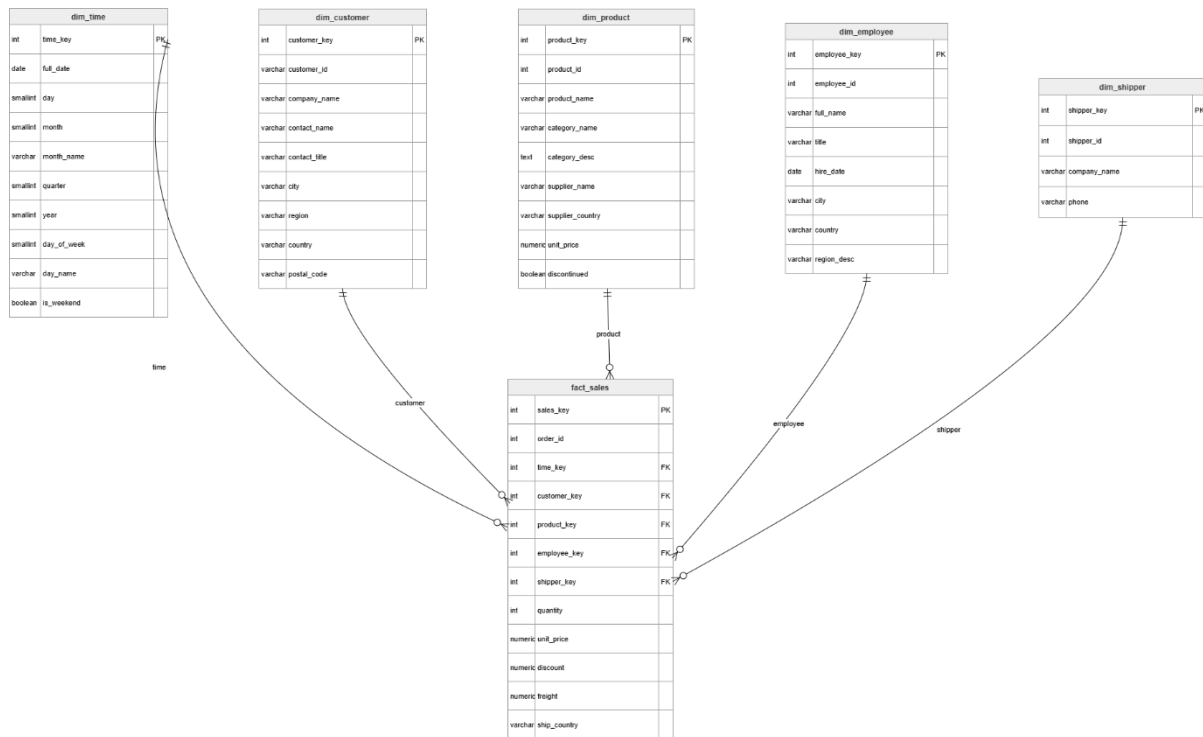
Tabela faktów: fact_sales

Zawiera miary biznesowe - wartości liczbowe, które są przedmiotem analiz:

- unit_price - cena jednostkowa
- quantity - ilość
- discount - rabat
- net_revenue - przychód netto (kolumna wyliczana: $\text{unit_price} \times \text{quantity} \times (1 - \text{discount})$)
- Klucze obce do wszystkich tabel wymiarowych

Tabele wymiarowe:

Wymiar	Zawartość	Kluczowe atrybuty
dim_time	Wymiar czasu	rok, kwartał, miesiąc, dzień tygodnia
dim_customer	Klienci	company_name, contact_name, city, country
dim_product	Produkty	product_name, category_name, supplier_name
dim_employee	Pracownicy	first_name, last_name, title, country
dim_shipper	Przewoźnicy	company_name, phone



4.1. Implementacja

Krokami transformacji były:

1. Generowanie klucza czasu (dim_time)
2. Denormalizacja wymiaru produktu: tabele products, categories i suppliers zmienione w jedną tabelę dim_product.
3. Denormalizacja wymiaru pracownika : połączono employees, employee_territories, territories i region w jeden wiersz na pracownika z przypisaną nazwą regionu.
4. Klucze zastępcze (surrogate keys): każda tabela wymiarowa otrzymała klucz sekwencyjny (SERIAL/AUTOINCREMENT) jako klucz główny.
5. Czyszczenie danych: wartości NULL w kolumnie order_date były filtrowane przy ładowaniu dim_time.

Tabela faktów:

```
CREATE TABLE fact_sales (  
    sales_key      SERIAL PRIMARY KEY,  
    order_id       SMALLINT NOT NULL,  
    time_key       INT NOT NULL REFERENCES dim_time(time_key),  
    customer_key   INT NOT NULL REFERENCES dim_customer(customer_key),  
    product_key    INT NOT NULL REFERENCES dim_product(product_key),  
    employee_key   INT REFERENCES dim_employee(employee_key),  
    shipper_key    INT REFERENCES dim_shipper(shipper_key),  
    -- MIARY  
    quantity       SMALLINT NOT NULL,  
    unit_price     REAL NOT NULL,  
    discount       REAL NOT NULL DEFAULT 0,  
    freight        REAL,  
    gross_revenue  REAL GENERATED ALWAYS AS  
                    (unit_price * quantity) STORED,  
    net_revenue    REAL GENERATED ALWAYS AS  
                    (unit_price * quantity * (1 - discount)) STORED,  
    ship_country   VARCHAR(15)  
);
```

Przykładowa tabela wymiaru:

```
CREATE TABLE dim_product (  
    product_key    SERIAL PRIMARY KEY,  
    product_id     SMALLINT NOT NULL,    -- klucz OLTP  
    product_name   VARCHAR(40) NOT NULL,  
    category_name  VARCHAR(15) NOT NULL,  
    category_desc  TEXT,  
    supplier_name  VARCHAR(40),  
    supplier_country VARCHAR(15),  
    unit_price     REAL,  
    discontinued   INTEGER NOT NULL  
);
```

Następnie dane wczytane ze skryptu *northwind.sql* zostały wczytane za pomocą zapytań takich jak:

```
INSERT INTO fact_sales (order_id, time_key, customer_key,  
                        product_key, employee_key, shipper_key,  
                        quantity, unit_price, discount,  
                        freight, ship_country)  
  
SELECT  
    o.order_id,  
    dt.time_key,
```

```

    dc.customer_key,
    dp.product_key,
    de.employee_key,
    ds.shipper_key,
    od.quantity,
    od.unit_price,
    od.discount,
    o.freight,
    o.ship_country
FROM order_details od
JOIN orders      o  ON od.order_id  = o.order_id
JOIN dim_time    dt ON dt.full_date = o.order_date
JOIN dim_customer dc ON dc.customer_id = o.customer_id
JOIN dim_product dp ON dp.product_id = od.product_id
LEFT JOIN dim_employee de ON de.employee_id = o.employee_id
LEFT JOIN dim_shipper ds ON ds.shipper_id = o.ship_via;

```

W wyniku ładowania otrzymano:

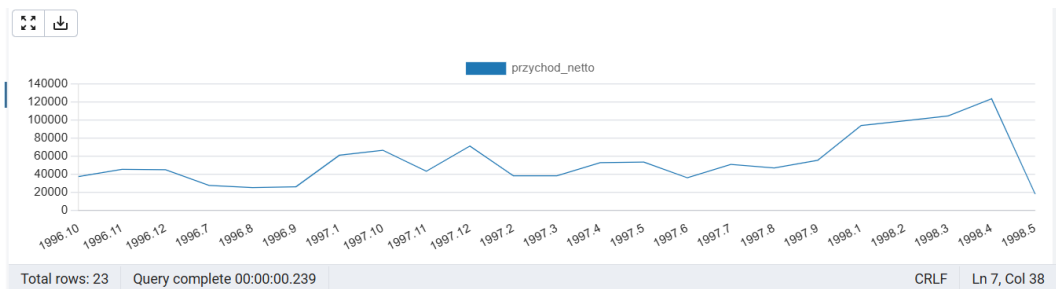
Tabela	Liczba wierszy
dim_time	480
dim_customer	91
dim_product	77
dim_employee	9
dim_shipper	6
fact_sales	2 155

Kluczowym testem weryfikacyjnym było porównanie sum przychodów między OLTP a DWH:

System	Przychód netto (suma)
OLTP (order_details)	\$1 265 793,04
DWH (fact_sales)	\$1 265 793,04

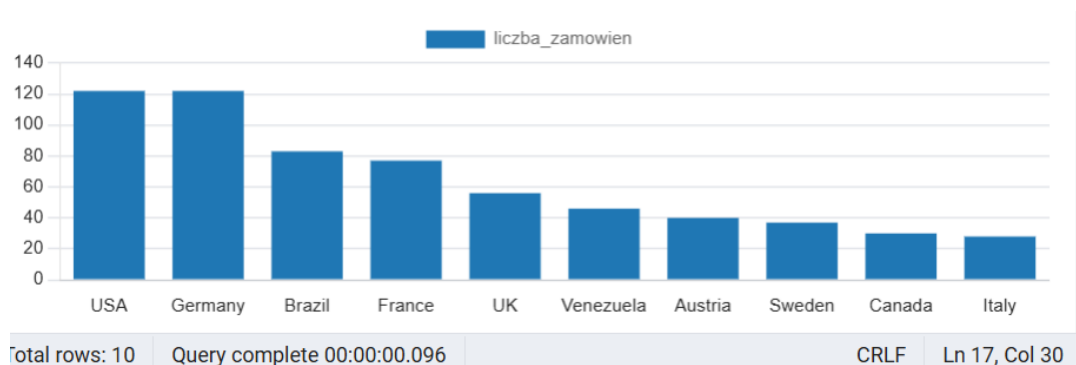
4.2. Analiza Danych – Zapytania SQL

1. Trend sprzedaży miesięcznej



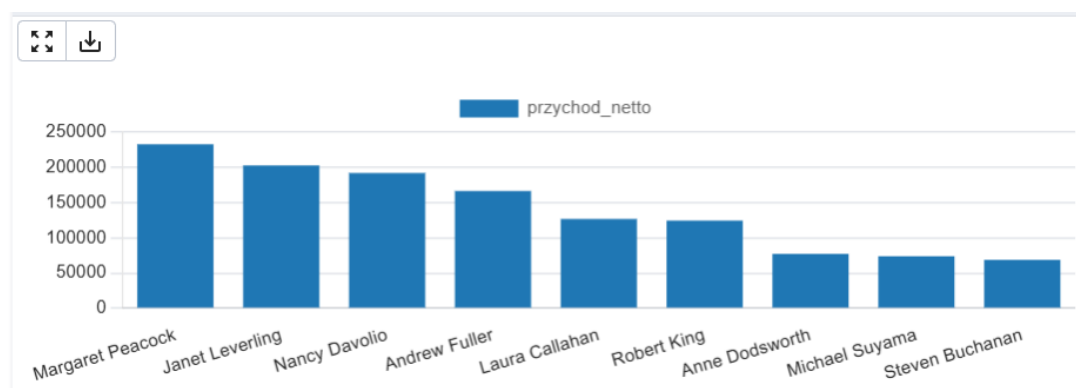
Dane obejmują okres od lipca 1996 do maja 1998. Rok 1997 jest jedynym pełnym rokiem operacyjnym. Wyraźne są sezonowe szczyty sprzedaży w miesiącach wrzesień-grudzień.

2. Porównanie liczby zamówionych produktów według krajów



Największą liczbę zamówień złożyły USA i Niemcy (po ~122 zamówienia), znacznie wyprzedzając pozostałe rynki. Na kolejnych miejscach znalazły się Brazylia (~83) i Francja (~77). Działania sprzedażowe i marketingowe powinny być skoncentrowane na rynkach USA i Niemiec w celu utrzymania zamówień a następnie włączać pozostałe kraje.

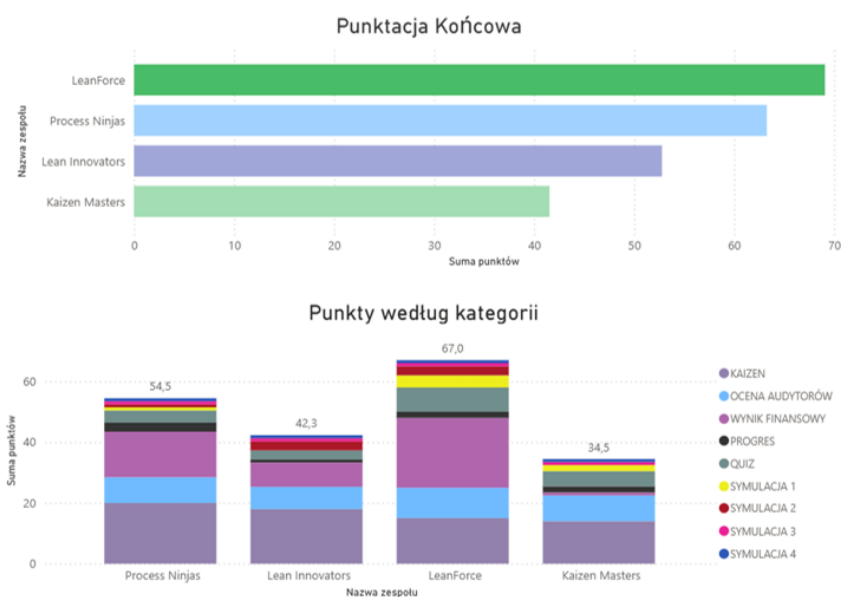
3. Wyniki pracowników – przychód netto



Margaret Peacock osiągnęła najwyższy przychód (\$232 891, 156 zamówień). Trzech pracowników (Peacock, Leverling, Davolio) odpowiada za ponad 49% całkowitego przychodu.

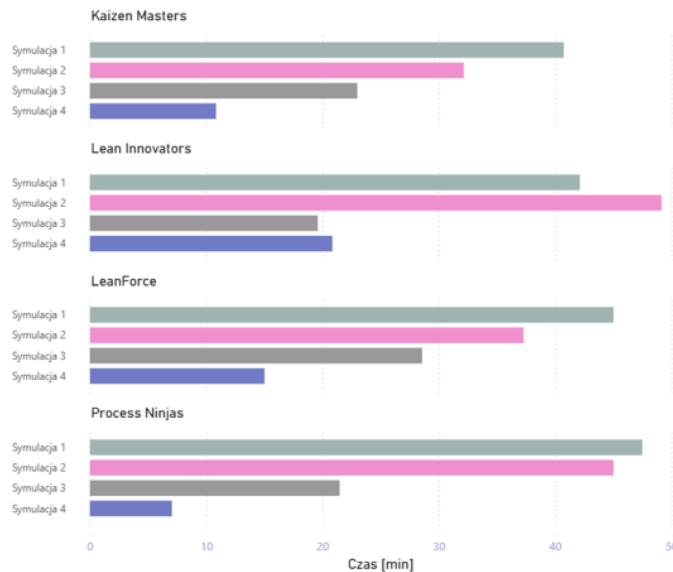
Kluczowe wnioski biznesowe z analizy danych

- Rynek: USA i Niemcy generują 38% przychodów; Austria wyróżnia się wysoką wartością transakcji.
- Pracownicy: Troje pracowników spośród dziewięciu obsługuje niemal połowę całego przychodu.



Analiza Postępu i Czasów Realizacji Symulacji

Porównanie czasu trwania poszczególnych symulacji dla zespołów (minuty)

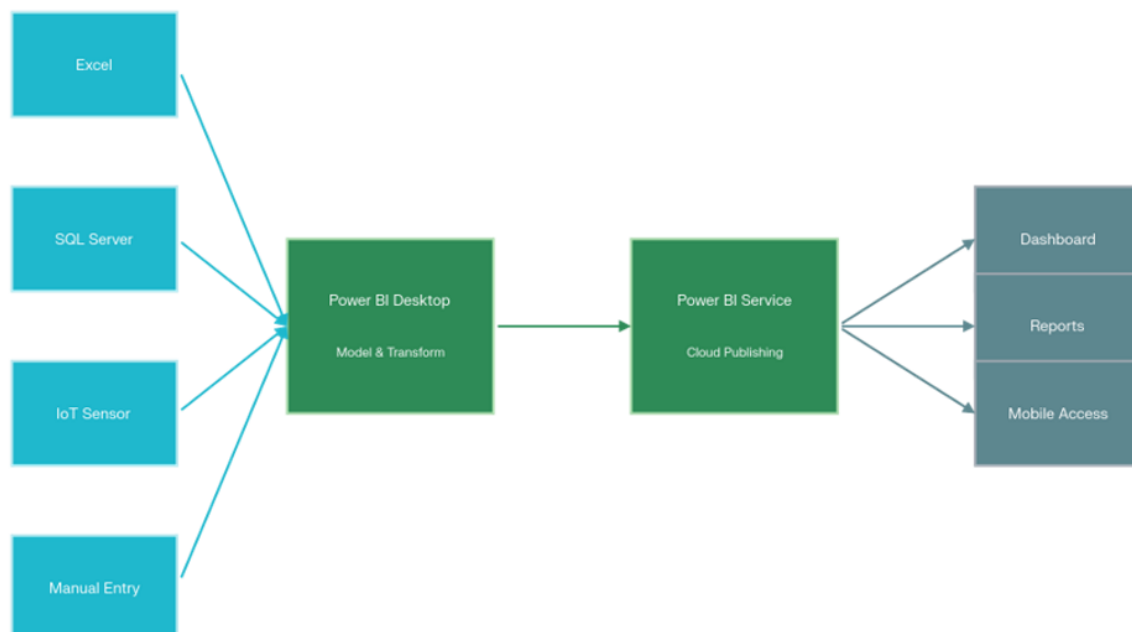


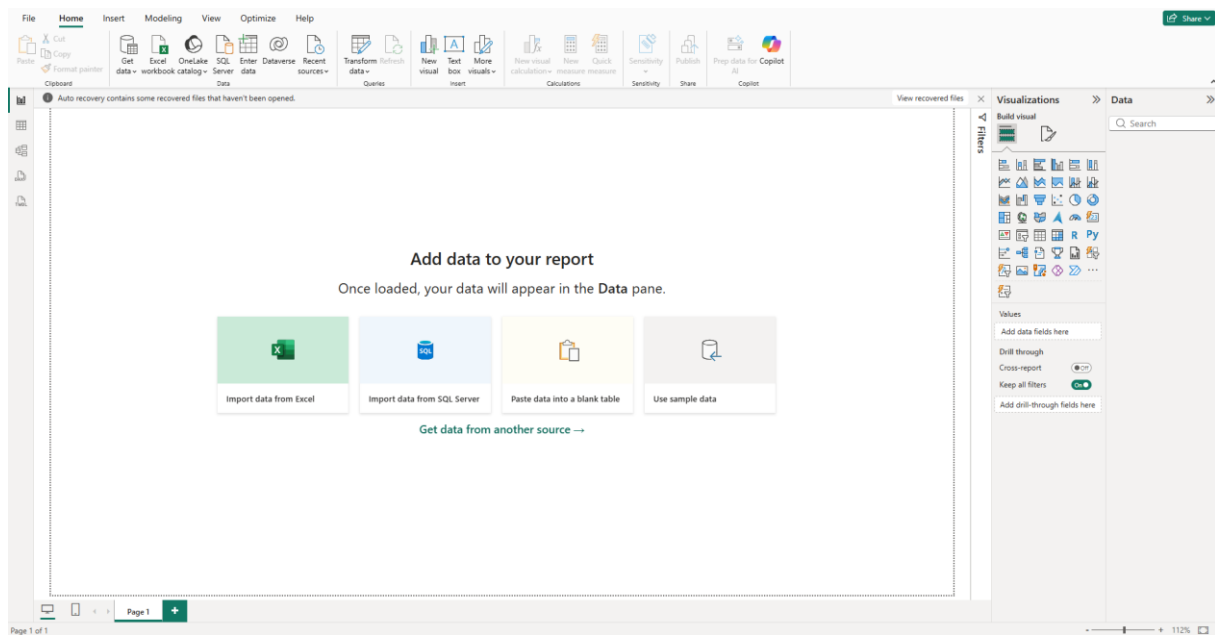
NAZWA ZESPOŁU	% poprawa 1→2	% poprawa 2→3	% poprawa 3→4
Kaizen Masters	21,10%	28,46%	52,83%
LeanForce	17,19%	23,36%	47,43%
Process Ninjas	5,22%	52,31%	67,16%
Lean Innovators	-16,69%	60,12%	-6,38%
Total	6,72%	43,37%	41,96%

(Tekst i wizualizacje zaczerpnięte z referatu pt. „Przejrzystość jako strategia:

Jak Power BI wspiera lean management” autorstwa Karoliny Murzyn i Adama Bujaka. Wygłoszonego podczas 66. Barbórkowej Konferencji Studenckich Kół Naukowych AGH)

Power BI realizuje przetwarzanie danych w czterech warstwach: źródła danych (CSV, SQL, Excel), Power Query (ETL - transformacja i łączenie tabel), model danych z miarami DAX, interaktywny dashboard dostępny dla odbiorcy .





5.2. Tworzenie Tabeli Zdenormalizowanej

Denormalizacja to proces odwrotny do normalizacji polega na celowym łączeniu wielu znormalizowanych tabel w strukturę zawierającą wszystkie potrzebne dane w jednym miejscu.

W projekcie denormalizacja polegała na połączeniu 6 tabel OLTP w jedną tabelę **orders_denormalized**, która zawiera pełne informacje o każdej pozycji zamówienia (bez konieczności odwoływania się do innych tabel).

Do programu zostały zładowane pliki w formacie .csv (uprzednio wyeksportowanych z DBMS), jednak równie dobrze można było stworzyć połączenie między bazą danych a programem.

Transformacja została dokonana w narzędziu Power Query:

- Ujednolicenie typów danych.
- Denormalizacja: Połączono 6 tabel w jedną denormalizowaną tabelę Wykonano 5 złączeń: order_details + orders + customers + products + categories + employees.
- Dodanie wyliczonych kolumn: np. przychód netto
- Agregacje (Group By): Sprzedaż po produkcie, zamówienia na klienta, sprzedaż po kategorii,
- Usunięcie zbędnych kolumn

```

let
od = order_details,
o = orders,
c = customers,
p = products,
cat = categories,
emp = employees,

// Merge 1: order_details + orders
M1 = Table.NestedJoin(od, "order_id", o, "order_id", "orders", JoinKind.LeftOuter),
E1 = Table.ExpandTableColumn(M1, "orders",
{"customer_id","employee_id","order_date","freight","ship_country"}),

// Merge 2: + customers (customer_id = Text po obu stronach)
E1fix = Table.TransformColumnTypes(E1, {"customer_id", type text}),
c2 = Table.TransformColumnTypes(c, {"customer_id", type text}),
M2 = Table.NestedJoin(E1fix, "customer_id", c2, "customer_id", "customers", JoinKind.LeftOuter),
E2 = Table.ExpandTableColumn(M2, "customers",
{"company_name","contact_name","contact_title","city","country"}),

// Merge 3: + products (ujednoczenie product_id)
E2fix = Table.TransformColumnTypes(E2, {"product_id", type text}),
p2 = Table.TransformColumnTypes(p, {"product_id", type text}, {"category_id", type text}),
M3 = Table.NestedJoin(E2fix, "product_id", p2, "product_id", "products", JoinKind.LeftOuter),
E3 = Table.ExpandTableColumn(M3, "products",
{"product_name","category_id"}),

// Merge 4: + categories (ujednoczenie category_id)
E3fix = Table.TransformColumnTypes(E3, {"category_id", type text}),
cat2 = Table.TransformColumnTypes(cat, {"category_id", type text}),
M4 = Table.NestedJoin(E3fix, "category_id", cat2, "category_id", "categories", JoinKind.LeftOuter),
E4 = Table.ExpandTableColumn(M4, "categories",
{"category_name"}),

// Merge 5: + employees
M5 = Table.NestedJoin(E4, "employee_id", emp, "employee_id", "employees", JoinKind.LeftOuter),
E5 = Table.ExpandTableColumn(M5, "employees",
{"first_name","last_name"}),

// Konwersja typów liczbowych
FixTypes = Table.TransformColumnTypes(E5, {
{"unit_price", type number},
{"quantity", type number},
{"discount", type number},
{"freight", type number}
}),

// Kolumna wyliczana: net_revenue
AddRevenue = Table.AddColumn(FixTypes, "net_revenue",
each [unit_price] * [quantity] * (1 - [discount]),
type number),

// Kolumna wyliczana: employee_name
AddEmpName = Table.AddColumn(AddRevenue, "employee_name",
each [first_name] & " " & [last_name],

```

```
type text),

// Usunięcie zbędnych kolumn pomocniczych
Final = Table.RemoveColumns(AddEmpName,
{"first_name","last_name","category_id","employee_id"})
in
Final
```

Transformacje można wykonać ręcznie za pomocą wbudowanych funkcji i interfejsu graficznego Power Query (przyciski Merge, Group By, Add Column), co pozwala na prostsze korzystanie osobom bez wiedzy technicznej.

×

Merge

Select tables and matching columns to create a merged table.

categories

category_id	category_name	description	picture
1	Beverages	Soft drinks, coffees, teas, beers, and ales	\x
2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings	\x
3	Confections	Desserts, candies, and sweet breads	\x
4	Dairy Products	Cheeses	\x
5	Grains/Cereals	Breads, crackers, pasta, and cereal	\x

categories

category_id	category_name	description	picture
1	Beverages	Soft drinks, coffees, teas, beers, and ales	\x
2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonings	\x
3	Confections	Desserts, candies, and sweet breads	\x
4	Dairy Products	Cheeses	\x
5	Grains/Cereals	Breads, crackers, pasta, and cereal	\x

Join Kind

Left Outer (all from first, matching from second)

☐ Use fuzzy matching to perform the merge

▸ Fuzzy matching options

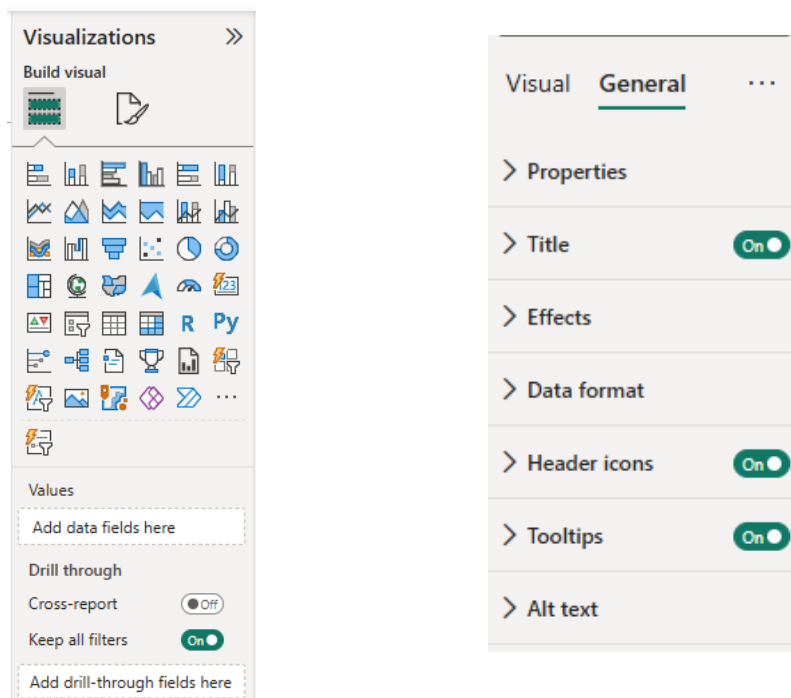
OK Cancel

W projekcie bezpośrednio pisany był kod w Advanced Editor, co pozwoliło na wykonanie wszystkich złączeń i transformacji w jednym bloku kodu. Skróciło to czas pracy i wyeliminowało ryzyko błędów wynikających z ręcznego klikania kolejnych kroków. Dodatkową zaletą jest fakt iż kod można w każdej chwili odtworzyć lub zmodyfikować.

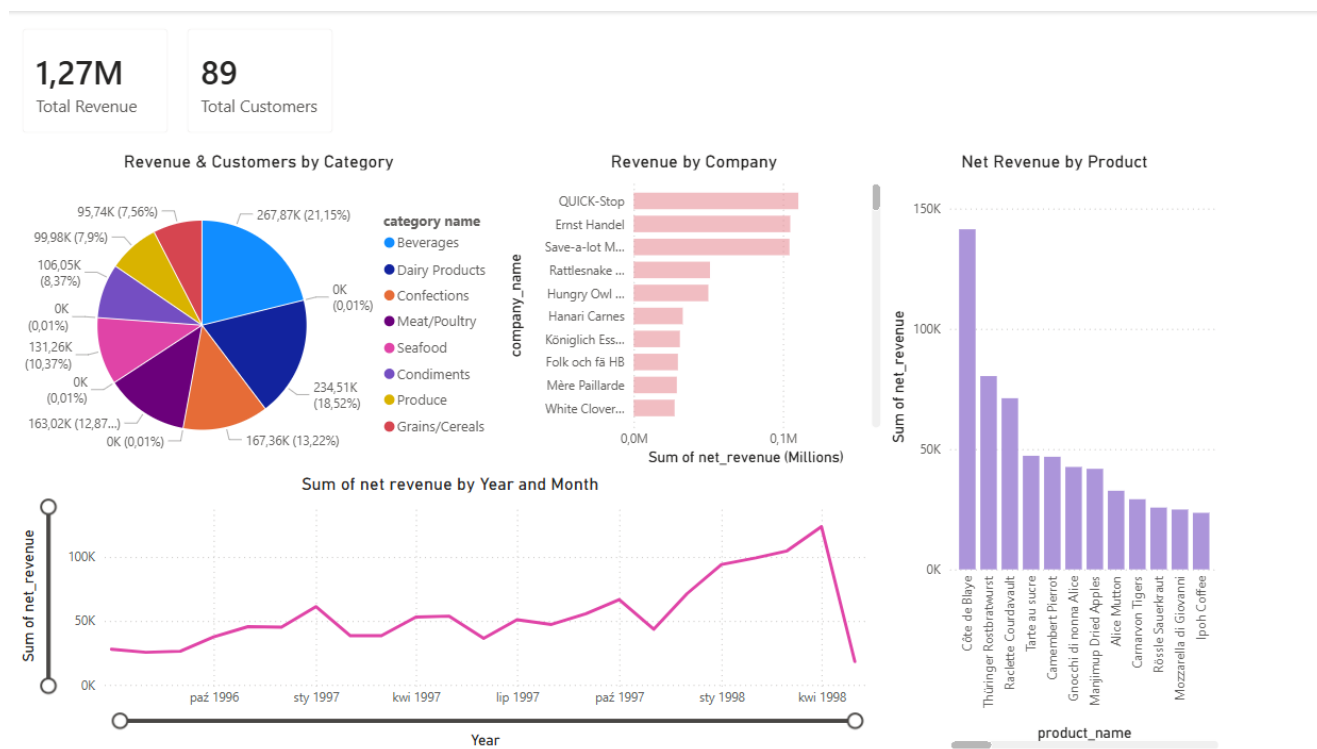
5.3.Wizualizacje

Na podstawie tabeli zdenormalizowanej możliwe jest tworzenie różnorodnych wizualizacji dzięki opcjom panelu Visualizations w Power BI. Narzędzie oferuje szeroki wachlarz typów wykresów m.in. wykresy słupkowe i kolumnowe, liniowe, warstwowe, kołowe i pierścieniowe, mapy, macierze, tabele, karty KPI etc.

Każdą wizualizację można dowolnie konfigurować w panelu Format, dostosowując tytuł, kolory, formatowanie danych, ikony nagłówków oraz tekst alternatywny.

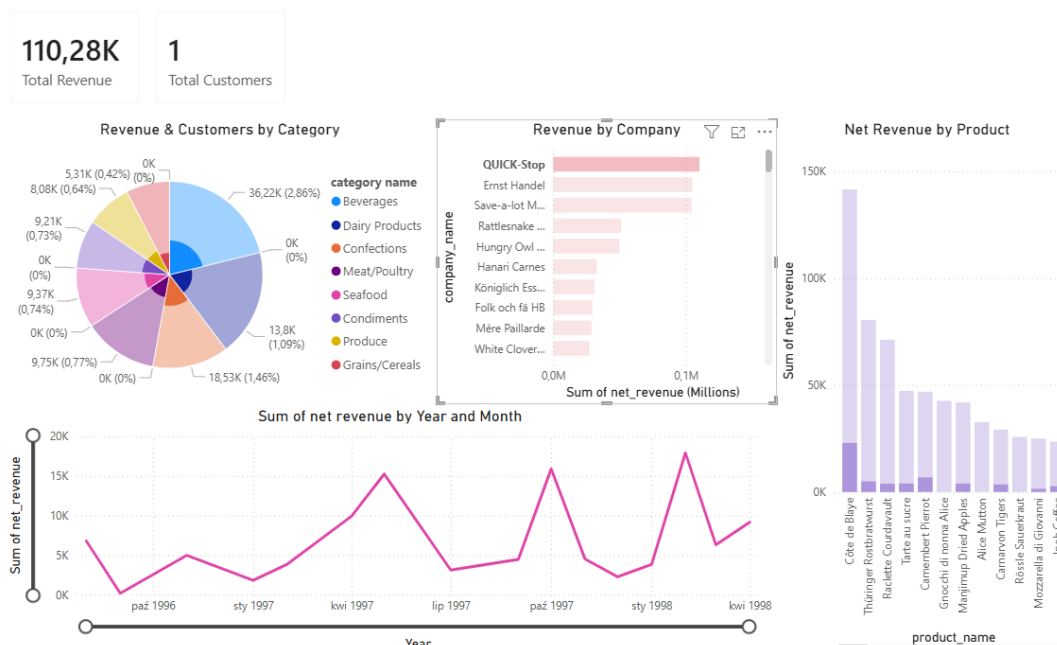
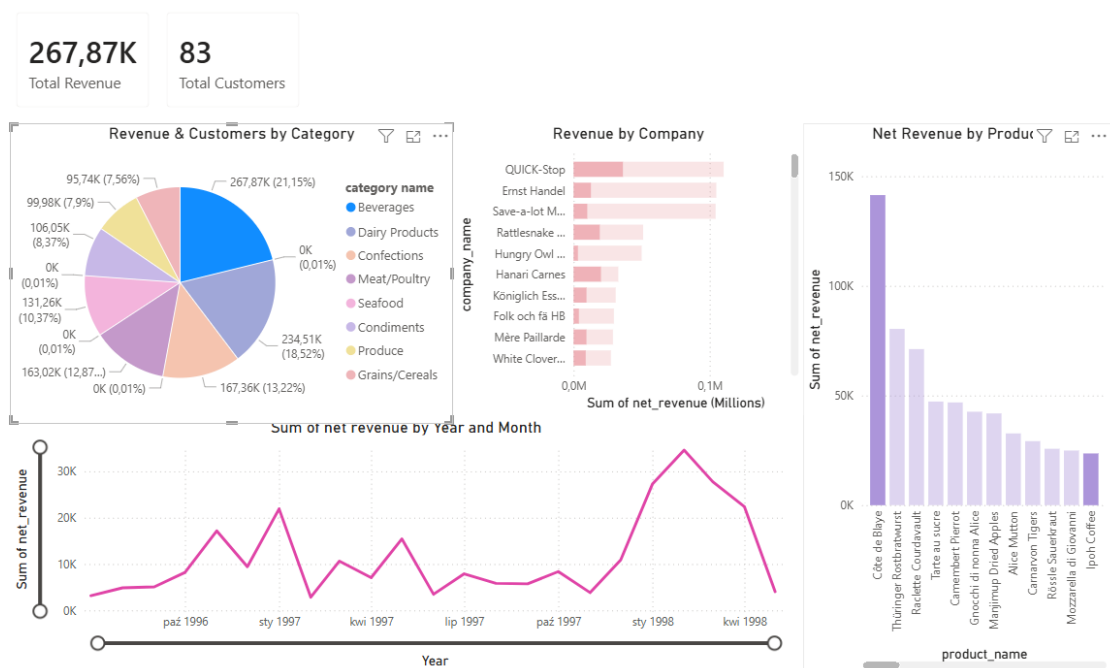


Przykładowy dashboard:



Szczególnie istotną funkcją Power BI jest interaktywność między wizualizacjami. Kliknięcie na dowolny element wykresu, np. słupek reprezentujący konkretną firmę klienta lub wycinek wykresu kołowego dla danej kategorii produktu, powoduje automatyczne filtrowanie wszystkich pozostałych wizualizacji na stronie raportu.

Elementy niezwiązane z wybraną wartością zostają wyszarzone. Dzięki temu analityk może w ciągu jednego kliknięcia odpowiedzieć na pytanie: Jaki był trend sprzedaży tej konkretnej firmy? Które produkty zamawiała? Przez którego pracownika były obsługiwane jej zamówienia?



6. Wnoski

Schemat gwiazdy upraszcza analizę, zapytania są krótsze i czytelniejsze niż ich odpowiedniki na bazie OLTP, zamiast łączyć kilka tabel, wystarczy połączenie do wymiaru.

Czyszczenie danych jest krytyczne w projekcie, brak zgodnych typów danych może doprowadzić do błędów i opóźnień.

Power BI i SQL to uzupełniające się narzędzia (możliwość bezpośredniego połączenia DBMS i Power BI), obydwa podejścia mogą prowadzić do tych samych wniosków różnią się jednak czasochłonnością, poziomem zaawansowania oraz nastawieniem („chłodne” w odbiorze wyniki zapytań kontra graficzność dashboardu).

Z perspektywy odbiorcy dashboardu Power BI oznacza:

- System liczy, zespół działa.
Odbiorca może samodzielnie eksplorować dane, zadawać pytania i wyciągać wnioski bez pośrednictwa analityka/działu IT .
- Jedna tablica. Każdy ekran. Aktualne dane.
Wszyscy interesariusze widzą te same, spójne informacje w czasie rzeczywistym.

Z perspektywy osoby budującej raport:

- Pełna transparentność
Struktura danych i logika obliczeń są widoczne i edytowalne.
- Nowoczesny standard raportowania
Wzmacnia profesjonalny odbiór analiz.
- Dane pod ochroną
Kontrola dostępu na poziomie raportu i wiersza danych

(Zaczerpnięte z referatu pt. „Przejrzystość jako strategia:

Jak Power BI wspiera lean management” autorstwa Karoliny Murzyn i Adama Bujaka. Wygłoszonego podczas 66. Barbórkowej Konferencji Studenckich Kół Naukowych AGH)

W pracy wykorzystano narzędzia generatywnego AI w celu wygenerowania kodu użytego w Power Query.